Create a complete, professional, production-ready student e-commerce marketplace application called "PasaBuy".

IMPORTANT:

Do not create a simple CRUD demo. Build this as a polished, real-world marketplace application with a clean, friendly, modern UI that students would actually want to use.

==========

1. PROJECT NAME

==========

Application Name:

PasaBuy

Tagline:

"Your Campus. Your Marketplace."

Purpose:

PasaBuy is a student-focused marketplace where students can buy and sell items within their school/community.

Students can:

- Browse products
- Search products
- Sell products
- Chat with sellers
- Save products
- Create Wanted posts
- Respond to Wanted posts
- Reserve products

- Meet face-to-face
- Complete transactions
- Review each other
- Report users/listings

IMPORTANT BUSINESS MODEL:

PasaBuy does NOT process the actual product payment.

The buyer and seller arrange the actual transaction themselves and meet face-to-face.

Example:

Seller posts:

Scientific Calculator

Selling Price: ₱500

PasaBuy posting fee:

₱5

Seller pays PasaBuy ₱5 through PayMongo.

After payment is confirmed:

The listing becomes ACTIVE.

Later:

Buyer meets seller.

Buyer gives seller ₱500 directly.

Seller gives buyer the calculator.

PasaBuy does NOT collect the ₱500.

PasaBuy only earns money from listing/posting fees.

==========

2. TECHNOLOGY STACK

==========

Mobile Application:

.NET MAUI

C#

XAML

MVVM

Backend:

ASP.NET Core Web API

Database:

MySQL

Admin:

Separate ASP.NET Core Web Admin Portal

Payment:

PayMongo

Authentication:

JWT / secure token-based authentication

Architecture:

Clean, modular, scalable, maintainable, secure.

Use Dependency Injection.

Use async/await.

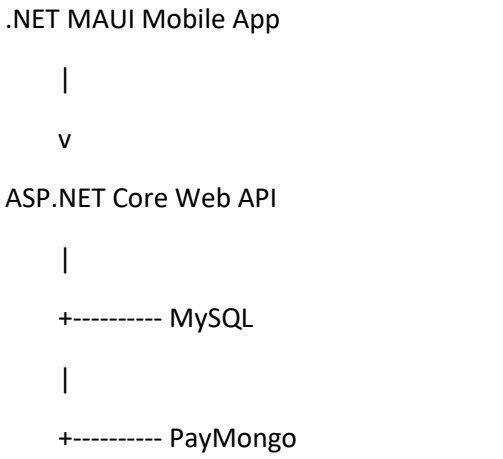
Do not block the UI thread.

==========

3. APPLICATION ARCHITECTURE

==========

Architecture:



Separate:

ASP.NET Core Admin Web Portal

|

v

ASP.NET Core Web API

IMPORTANT:

Never place PayMongo secret keys inside the .NET MAUI application.

Correct:

.NET MAUI

↓

ASP.NET Core API

↓

PayMongo

Never:

.NET MAUI

↓

PayMongo Secret Key

All PayMongo secret credentials must remain on the backend.

Use environment variables or secure server configuration.

Never commit API keys to source control.

==========

4. USER ROLES

==========

There are two major roles:

1. STUDENT

2. ADMIN

A Student can be both a buyer and seller.

There is no need for separate buyer and seller accounts.

A student can:

- Buy

- Sell

- Create Wanted posts

- Respond to Wanted posts

Admins use a completely separate web application.

==========

5. STUDENT APP NAVIGATION

==========

Use a modern bottom navigation bar:

Home

Explore

Sell

Wanted

Messages

Profile

Make navigation smooth and intuitive.

==========

6. UI / DESIGN REQUIREMENTS

==========

The app must look professional and modern.

Design style:

- Clean
- Friendly
- Student-focused
- Modern
- Minimal
- Professional
- Easy to understand
- Fast
- Visually attractive

Use:

- Rounded cards
- Rounded buttons
- Clean typography

- Proper spacing
- Product image cards
- Soft shadows
- Clear icons
- Consistent UI components
- Friendly empty states
- Loading skeletons
- Pull-to-refresh
- Subtle animations

Avoid:

- Excessive gradients
- Excessive animations
- Crowded screens
- Tiny buttons
- Too many colors
- Complicated navigation
- Old-looking UI
- Generic CRUD-style screens

The UI should feel like a real commercial marketplace.

==========

7. BRAND

==========

Name:

PasaBuy

Tagline:

"Your Campus. Your Marketplace."

Secondary message:

"Buy what you need. Sell what you don't. Find what you're looking for."

Brand personality:

- Friendly
- Trustworthy
- Young
- Local
- Student-oriented
- Safe
- Convenient

Create a clean PasaBuy logo/icon.

==========

8. LOGIN SCREEN

==========

Create a professional login screen.

Include:

- PasaBuy logo

- Welcome message
- Email
- Password
- Show/hide password
- Remember me
- Login button
- Forgot password
- Create account

Text:

"Welcome back"

"Log in to continue to PasaBuy."

Validation:
- Required fields
- Valid email
- Correct credentials

Show friendly error messages.

==========

9. REGISTRATION

==========

Registration fields:

- First Name

- Last Name
- Student Number
- School Email
- Password
- Confirm Password
- Course/Program
- Year Level
- Profile Picture

Require:

Terms and Conditions

Privacy Policy

Validation:

- Required fields
- Valid email
- Unique email
- Unique student number
- Password strength
- Password confirmation
- Valid student information

==========

10. STUDENT VERIFICATION

==========

Account statuses:

UNVERIFIED

PENDING

VERIFIED

SUSPENDED

Verified students display:

✓ Verified Student

The badge should appear on:

- Seller profile
- Product listing
- Product details
- Chat/product information

==========

11. HOME SCREEN

==========

Home screen should contain:

"Good morning, [Name]"

Search bar:

"Search for products..."

Sections:

Categories

Recommended for You

Recently Listed

Wanted Near You

Popular Items

Product cards should show:

- Product image
- Product name
- Price
- Condition
- Approximate distance
- Favorite button
- Verified seller badge

==========

12. CATEGORIES

==========

Initial categories:

Electronics

Gadgets

School Supplies

Books

Clothing

Shoes

Accessories

Dorm Items

Sports

Beauty

Food

Other

Admins can:

- Add categories
- Edit categories
- Disable categories
- Reorder categories

==========

13. EXPLORE

==========

Explore screen:

- Search
- Categories
- Product grid/list
- Filters

- Sorting

Filters:

- Category
- Minimum price
- Maximum price
- Condition
- Distance
- Verified seller

Sorting:

- Newest
- Oldest
- Lowest Price
- Highest Price
- Most Viewed

Use pagination or infinite scrolling.

==========

14. PRODUCT DETAILS

==========

Product details screen should contain:

Multiple product images.

Allow:

- Swipe images
- Full screen image viewing
- Zoom where possible

Show:

Product Name

Price

Condition

Description

Category

Date Posted

Approximate Location

Availability

Seller section:

- Profile picture
- Seller name
- Verified badge
- Rating
- Completed transactions
- Number of reviews

Buttons:

Message Seller

Save

Share

Report

IMPORTANT:

Do NOT create a traditional "Buy Now" payment button.

PasaBuy does not process the actual product payment.

==========

15. SELL PRODUCT

==========

Create a multi-step selling process.

STEP 1:

Upload photos.

Allow up to 8 images.

Sources:

- Camera
- Gallery

Require at least one image.

Compress images before upload.

==========

16. PRODUCT INFORMATION

==========

Fields:

Product Name

Category

Description

Condition

Price

Condition options:

New

Like New

Good

Fair

Used

==========

17. PRICE

==========

Seller enters the selling price.

Example:

Selling Price:

₱500

The app can display the estimated PasaBuy posting fee.

IMPORTANT:

The backend MUST calculate the final posting fee.

Never trust a fee sent from the mobile application.

==========

18. POSTING FEE RULE

==========

Use exactly this pricing:

₱1–₱99 = ₱1 posting fee

₱100–₱999 = ₱5 posting fee

₱1,000 and above = ₱10 posting fee

Examples:

₱50 → ₱1

₱99 → ₱1

₱100 → ₱5

₱500 → ₱5

₱999 → ₱5

₱1,000 → ₱10

₱10,000 → ₱10

The backend must calculate this.

Create a reusable FeeCalculationService.

==========

19. MEETUP LOCATION

==========

Seller chooses an approved meetup location.

Examples:

School Gate

Library

Cafeteria

Student Plaza

Gym

Designated Marketplace Area

IMPORTANT:

Do NOT publicly show the seller's home address.

Only allow safe public meetup locations.

==========

20. LISTING PREVIEW

==========

Before publishing, show:

LISTING PREVIEW

Product image

Product Name:

Scientific Calculator

Selling Price:

₱500

Condition:

Good

Description:

...

Meetup:

School Library

Selling Price:

₱500

PasaBuy Posting Fee:

₱5

PAY ₱5 & POST

==========

21. PAYMONGO PAYMENT

==========

PayMongo is used ONLY for PasaBuy posting fees.

Preferred payment experience:

Use PayMongo's supported QR/payment flow where available.

Payment process:

Seller creates listing

↓

Backend calculates posting fee

↓

Backend creates PayMongo payment

↓

App displays PayMongo payment/QR

↓

Seller pays

↓

PayMongo processes payment

↓

PayMongo sends webhook

↓

Backend verifies payment

↓

Payment becomes PAID

↓

Listing becomes ACTIVE

==========

22. PAYMENT SCREEN

==========

Create a clean payment screen.

Title:

"Pay Posting Fee"

Show:

Your Listing

Scientific Calculator

Selling Price:

₱500

Posting Fee:

₱5

Then:

"Scan the QR code to pay ₱5"

Display the PayMongo-supported QR/payment interface.

Show:

Waiting for payment...

Cancel

The user must NOT be able to manually tell the application:

"I already paid."

Payment status must come from the backend.

==========

23. PAYMONGO SECURITY

==========

IMPORTANT:

Never put PayMongo secret credentials in the mobile application.

The backend communicates with PayMongo.

Use:

PayMongoService

PaymentService

Webhook handling.

Use secure environment variables.

Never hard-code:

API secret keys

Webhook secrets

Private credentials

==========

24. PAYMENT STATUSES

==========

Use:

PENDING

PAID

FAILED

CANCELLED

REFUNDED

Only PAID payments can activate listings.

==========

25. PAYMONGO WEBHOOK

==========

Create a backend webhook endpoint.

The webhook must:

1. Receive PayMongo event
2. Validate/authenticate the event where supported
3. Check event ID
4. Prevent duplicate processing
5. Find the related payment
6. Verify payment status
7. Update payment
8. Activate listing if successful
9. Create notification
10. Record webhook event

Webhook events must be idempotent.

==========

26. LISTING STATUS

==========

Use:

DRAFT

PENDING_PAYMENT

ACTIVE

RESERVED

SOLD

EXPIRED

REMOVED

REJECTED

Only ACTIVE listings appear publicly.

==========

27. AFTER PAYMENT

==========

When payment is confirmed:

Show:

"Your listing is now live!"

Buttons:

View Listing

The seller receives notification:

"Your Scientific Calculator listing has been published successfully."

==========

28. PAYMENT FAILURE

==========

If payment fails:

Show:

"Payment wasn't completed."

Buttons:

Try Again

Cancel

Listing remains:

PENDING_PAYMENT

It must not appear publicly.

==========

29. MY LISTINGS

==========

Profile → My Listings

Tabs:

Active

Reserved

Sold

Draft

Pending Payment

Expired

Actions:

Edit

Pause

Delete

Reserve

Mark Sold

Renew

View

==========

30. WANTED FEATURE

==========

Students can post things they are looking for.

Example:

WANTED

Arduino Uno

Budget:

₱800

Description:

"Need an Arduino Uno for my school project."

Wanted fields:

Title

Description

Category

Maximum Budget

Condition

Image

Preferred Meetup

Expiration

==========

31. WANTED RESPONSE

==========

A seller can respond to a Wanted post.

Example:

"I have one available."

Offer:

₱650

Condition:

Good

The requester can then start a conversation with the seller.

Wanted statuses:

ACTIVE

FULFILLED

EXPIRED

CANCELLED

REMOVED

==========

32. CHAT SYSTEM

==========

Chat is a major feature.

A buyer can message a seller from any product listing.

Button:

Message Seller

When opening chat from a product, automatically attach/reference the product.

Example:

Scientific Calculator

~~₱500~~

Buyer:

"Hi! Is this still available?"

Seller:

"Yes, it is."

Buyer:

"Can we meet tomorrow?"

Seller:

"Sure. We can meet at the library."

==========

33. CHAT FEATURES

==========

Support:

- Text messages
- Images
- Product preview
- Wanted post preview
- Timestamps
- Read/unread status
- Notifications
- Block
- Report

If real-time messaging is implemented, support:

- Real-time messages
- Typing indicator
- Online status

Otherwise use reliable polling/refresh.

==========

34. MESSAGE LIST

==========

Messages screen:

Messages

Search

Example:

John D.

Scientific Calculator

"Yes, it's still available."

2m

Maria S.

iPhone Case

"Can meet tomorrow."

15m

Kevin R.

Arduino Uno

"₱650 is okay."

1h

Unread conversations must display an unread badge.

==========

35. CHAT SAFETY

==========

Include:

Block User

Report User

Safety message:

"Meet in a safe public location and inspect the item before paying."

Do not encourage sharing:

- Passwords
- OTPs
- Private credentials
- Banking credentials

==========

36. RESERVE ITEM

==========

Seller can select:

Reserve Item

Listing changes:

ACTIVE

↓

RESERVED

Other buyers should see:

Reserved

Other buyers cannot reserve the same listing.

Reservation must be enforced server-side.

==========

37. FACE-TO-FACE PURCHASE

==========

The actual purchase happens between buyer and seller.

Example:

Product:

Scientific Calculator

Price:

₱500

Buyer

↓

₱500 cash

↓

Seller

Seller

↓

Calculator

↓

Buyer

PasaBuy receives only:

₱5 posting fee.

PasaBuy does NOT receive:

₱500 product price.

==========

38. MARK AS SOLD

==========

Seller can select:

Mark as Sold

Confirmation:

"Did you successfully complete this transaction?"

Buttons:

Yes, Mark as Sold

Cancel

Listing changes:

RESERVED

↓

SOLD

==========

39. BUYER CONFIRMATION

==========

Buyer receives:

"Did you receive this item?"

Buttons:

Yes, I Received It

Report a Problem

After confirmation:

Transaction becomes:

COMPLETED

==========

40. TRANSACTION RECORD

==========

Even though PasaBuy does not process the product payment, create a transaction record.

Store:

Buyer

Seller

Listing

Agreed Price

Meetup Location

Date

Status

IMPORTANT:

Clearly indicate that the agreed price is not money collected by PasaBuy.

==========

41. REVIEWS

==========

After completed transactions, both sides can review each other.

Rating:

1–5 stars

Optional comment.

Example:

★★★★★

"Very easy to deal with."

Prevent:

- Self reviews
- Duplicate reviews
- Reviews without completed transactions

==========

42. FAVORITES

==========

Students can save products.

Product card:

♡ Save

Saved:

Favorites page shows:

Product

Price

Seller

Availability

Notify users when appropriate if:

- Product becomes sold
- Product is removed
- Price changes

==========

43. SEARCH

==========

Search should support:

- Product name
- Description
- Category
- Seller

Example:

Search:

calculator

Results:

Scientific Calculator

Casio Calculator

Engineering Calculator

==========

44. NOTIFICATIONS

==========

Notifications for:

- Successful posting payment
- Failed payment
- Listing published
- New message
- Wanted response
- Item reserved
- Item sold
- Buyer confirmation
- Review available
- Listing expiring
- Listing removed
- Account verification
- Report updates
- Admin announcements

==========

45. PROFILE

==========

Profile screen:

Profile Picture

Name:

[Student Name]

✓ Verified Student

Rating:

★★★★★ 4.9

Completed Transactions:

24

Options:

My Listings

Saved Items

Wanted Posts

Transactions

Reviews

Notifications

Settings

Help

Terms

Privacy

Logout

==========

46. SETTINGS

==========

Include:

Edit Profile

Change Password

Notification Settings

Privacy

Blocked Users

Theme

Delete Account

Logout

==========

47. REPORTING

==========

Users can report listings.

Reasons:

Scam

Fake Item

Prohibited Item

Misleading Description

Inappropriate Content

Stolen Item

Other

Users can report other users.

Reasons:

Scam

Harassment

Fake Identity

Suspicious Behavior

Other

==========

48. REPORT STATUS

==========

Use:

PENDING

INVESTIGATING

RESOLVED

REJECTED

Users can see the status of reports they submitted.

==========

49. BLOCKING

==========

Users can block other users.

Blocked users should not be able to:

- Start new conversations

- Continue normal interaction

- Contact the blocking user

Blocking must be enforced on the backend.

==========

50. PROHIBITED ITEMS

==========

Create an admin-managed prohibited-item system.

Possible prohibited categories:

- Illegal drugs

- Weapons

- Counterfeit products

- Stolen goods

- School-prohibited products

- Other restricted products

Do not rely only on mobile-side validation.

Backend must also validate listings.

==========

51. LOCATION

==========

Support:

- Approximate distance
- Nearby products
- Approved meetup locations

Do NOT publicly expose:

- Home address
- Exact private location

==========

52. ADMIN WEB PORTAL

==========

Create a completely separate web application for administrators.

Recommended:

ASP.NET Core MVC or Razor Pages.

The admin portal should be responsive and professional.

Admin users must not use the student mobile application for administration.

==========

53. ADMIN SIDEBAR

==========

PasaBuy Admin

Dashboard

Marketplace

- Listings
- Categories
- Wanted Posts

Users

- Students
- Suspended Users

Transactions

Payments

Reports

Analytics

Notifications

Meetup Locations

Settings

Audit Logs

Logout

==========

54. ADMIN DASHBOARD

==========

Dashboard cards:

Total Students

Verified Students

Active Listings

Today's Listings

Completed Transactions

Today's Revenue

Total Revenue

Pending Reports

Failed Payments

IMPORTANT:

Revenue means PasaBuy listing-fee revenue.

Do not describe product sale prices as PasaBuy revenue.

==========

55. ADMIN ANALYTICS

==========

Create charts for:

- Student registrations
- Listings
- Completed transactions
- Posting-fee revenue
- Popular categories
- Reports
- Payment success/failure

Filters:

Today

Last 7 Days

Last 30 Days

Last 12 Months

Custom Range

==========

56. ADMIN USER MANAGEMENT

==========

Admin can:

- Search students
- View profiles
- Verify students
- Suspend users
- Restore users
- View listings

- View transactions

- View reports

- View payment records

Never display passwords.

==========

57. ADMIN LISTING MANAGEMENT

==========

Admin can:

- Search listings

- Filter listings

- View listing

- Remove listing

- Restore listing

- Flag listing

- View seller

- View reports

- View payment record

When removing a listing, require a reason.

Notify the seller.

==========

58. ADMIN PAYMENT MANAGEMENT

==========

Payment page columns:

Payment ID

Student

Listing

Fee

Status

Date

Example:

PM-001 | Student A | Calculator | ₱5 | Paid | Aug 27

PM-002 | Student B | Shoes | ₱1 | Paid | Aug 27

PM-003 | Student C | Laptop | ₱10 | Failed | Aug 27

Show:

- Payment ID
- Listing
- Student
- Amount
- Currency
- PayMongo reference
- Status
- Created date
- Paid date

==========

59. ADMIN REPORT MANAGEMENT

==========

Admin can:

- View report
- View reported user
- View listing
- View reason
- Add internal notes
- Investigate
- Remove listing
- Suspend user
- Resolve report

==========

60. ADMIN CHAT PRIVACY

==========

Admins must NOT automatically have access to every private conversation.

Private conversation access should only be available when necessary for:

- User report
- Safety investigation
- Moderation requirement

Every administrative access to private conversations must be recorded in audit logs.

==========

61. ADMIN CATEGORY MANAGEMENT

==========

Admin can:

- Add category
- Edit category
- Disable category
- Reorder category
- Change category icon/image

Disabled categories cannot be selected for new listings.

==========

62. ADMIN MEETUP LOCATION MANAGEMENT

==========

Admin manages approved safe meetup locations.

Fields:

Name

Description

Location

Latitude

Longitude

Active Status

Examples:

School Gate

Library

Cafeteria

Student Plaza

Gym

==========

63. ADMIN SETTINGS

==========

Allow admin to configure:

- Posting fee brackets
- Listing expiration
- Maximum images
- Maximum image size
- Categories
- Meetup locations
- Notifications
- Prohibited categories
- Announcements

Business rules should be configurable where practical.

==========

64. AUDIT LOGS

==========

Record important administrative actions.

Examples:

Admin Login

User Suspended

User Restored

Listing Removed

Listing Restored

Report Resolved

Payment Viewed

Category Changed

Settings Changed

Private Conversation Accessed

Store:

Admin ID

Action

Target

Timestamp

Relevant Metadata

==========

65. DATABASE TABLES

==========

Use MySQL.

Recommended tables:

users

student_profiles

categories

listings

listing_images

listing_favorites

wanted_posts

wanted_responses

conversations

conversation_participants

messages

message_attachments

message_reads

blocked_users

transactions

reviews

reports

notifications

payments

payment_webhooks

listing_fees

meetup_locations

admin_users

audit_logs

platform_settings

==========

66. USERS TABLE

==========

Include:

id

email

password_hash

role

status

created_at

updated_at

last_login_at

Never store plaintext passwords.

==========

67. STUDENT PROFILES TABLE

==========

Include:

id

user_id

first_name

last_name

student_number

school_email

course

year_level

profile_image

verification_status

rating

completed_transactions

created_at

updated_at

==========

68. LISTINGS TABLE

==========

Include:

id

seller_id

category_id

title

description

price

condition

status

meetup_location_id

views

favorites_count

created_at

updated_at

expires_at

reserved_at

sold_at

==========

69. PAYMENTS TABLE

==========

Include:

id

user_id

listing_id

amount

currency

provider

provider_reference

status

created_at

paid_at

Provider should be:

PayMongo

==========

70. PAYMENT WEBHOOK TABLE

==========

Include:

id

provider

event_id

event_type

payload_reference

processed

processed_at

created_at

Prevent duplicate webhook processing.

==========

71. CHAT TABLES

==========

CONVERSATIONS:

id

listing_id

wanted_post_id

status

created_at

updated_at

CONVERSATION_PARTICIPANTS:

id

conversation_id

user_id

joined_at

MESSAGES:

id

conversation_id

sender_id

message_type

message_text

attachment_url

created_at

read_at

deleted_at

==========

72. TRANSACTIONS TABLE

==========

Include:

id

listing_id

buyer_id

seller_id

agreed_price

meetup_location_id

status

reserved_at

completed_at

created_at

Transaction statuses:

RESERVED

COMPLETED

CANCELLED

DISPUTED

==========

73. WANTED POSTS TABLE

==========

Include:

id

user_id

category_id

title

description

maximum_budget

condition

image_url

meetup_location_id

status

expires_at

created_at

updated_at

==========

74. REVIEWS TABLE

==========

Include:

id

transaction_id

reviewer_id

reviewed_user_id

rating

comment

created_at

Enforce:

One review per user per completed transaction.

==========

75. API ENDPOINTS

==========

AUTH:

POST /api/auth/register

POST /api/auth/login

POST /api/auth/refresh

POST /api/auth/forgot-password

POST /api/auth/reset-password

LISTINGS:

GET /api/listings

GET /api/listings/{id}

POST /api/listings

PUT /api/listings/{id}

DELETE /api/listings/{id}

POST /api/listings/{id}/reserve

POST /api/listings/{id}/sold

PAYMENTS:

POST /api/payments/listing

GET /api/payments/{id}

POST /api/payments/webhook

CHAT:

GET /api/conversations

GET /api/conversations/{id}

POST /api/conversations

POST /api/conversations/{id}/messages

WANTED:

GET /api/wanted

POST /api/wanted

GET /api/wanted/{id}

POST /api/wanted/{id}/respond

REVIEWS:

POST /api/reviews

GET /api/users/{id}/reviews

REPORTS:

POST /api/reports

GET /api/reports/my

FAVORITES:

POST /api/listings/{id}/favorite

DELETE /api/listings/{id}/favorite

BLOCKING:

POST /api/users/{id}/block

DELETE /api/users/{id}/block

NOTIFICATIONS:

GET /api/notifications

POST /api/notifications/{id}/read

Use proper authorization on every endpoint.

==========

76. MOBILE MVVM STRUCTURE

==========

Use:

Models

Views

ViewModels

Services

Repositories

Helpers

Converters

Controls

Resources

Use dependency injection.

Use clean separation of concerns.

==========

77. SERVICES

==========

Create:

AuthService

ListingService

CategoryService

PaymentService

PayMongoService

ChatService

NotificationService

UserService

ReviewService

ReportService

LocationService

UploadService

FavoriteService

WantedService

TransactionService

==========

78. VIEWMODELS

==========

Create:

LoginViewModel

RegisterViewModel

HomeViewModel

ExploreViewModel

ProductDetailsViewModel

SellViewModel

ListingPreviewViewModel

PaymentViewModel

WantedViewModel

WantedDetailsViewModel

MessagesViewModel

ChatViewModel

ProfileViewModel

MyListingsViewModel

TransactionsViewModel

SettingsViewModel

FavoritesViewModel

NotificationsViewModel

==========

79. LOADING STATES

==========

Never show blank screens while loading.

Use:

- Skeleton loaders
- Progress indicators
- Pull-to-refresh
- Proper loading buttons

Disable buttons when an operation is processing to prevent duplicate requests.

==========

80. EMPTY STATES

==========

NO LISTINGS:

"No products found."

NO FAVORITES:

"Nothing saved yet."

"Save products you like and find them here later."

NO MESSAGES:

"No conversations yet."

"Message a seller to start a conversation."

NO WANTED POSTS:

"Nothing here yet."

"Post something you're looking for."

==========

81. ERROR HANDLING

==========

Never expose raw technical errors to students.

Show:

"Something went wrong."

"Please try again."

Button:

Try Again

Log technical information securely on the backend.

==========

82. OFFLINE SUPPORT

==========

Detect internet connectivity.

When offline show:

"You're currently offline."

Allow safe local functionality such as:

- Viewing cached content
- Creating drafts

IMPORTANT:

Never pretend that:

- Payment succeeded
- Listing published
- Message delivered

unless the server confirmed it.

==========

83. ACCESSIBILITY

==========

Support:

- Good color contrast
- Large touch targets
- Accessible labels
- Screen reader support
- Dynamic text where practical
- Clear error messages

==========

84. PERFORMANCE

==========

Optimize:

- Image loading

- API requests
- Database queries
- Search
- Chat
- Caching
- Pagination

Use asynchronous operations.

Use image caching.

Compress uploaded images.

==========

85. SECURITY

==========

Implement:

- HTTPS
- JWT authentication
- Role-based authorization
- Server-side validation
- Rate limiting
- Secure file uploads
- SQL injection protection
- Ownership validation
- Secure password hashing
- Secure password reset

- Secure token handling

- Input validation

- API authorization

Never trust client-side values.

Example:

If mobile sends:

fee = ₱1

the backend must ignore the supplied fee and calculate the correct fee.

==========

86. PASSWORD SECURITY

==========

Never store plaintext passwords.

Use a strong password hashing algorithm.

Never return password hashes to clients.

Never log passwords.

Implement secure password reset.

==========

87. IMAGE SECURITY

==========

Validate:

- MIME type
- File extension
- File size
- Dimensions

Generate safe filenames.

Compress images.

Do not trust original filenames.

==========

88. PRODUCT OWNERSHIP

==========

Only the listing owner can:

- Edit listing
- Delete listing
- Reserve listing
- Mark listing sold

Backend must verify ownership.

Never rely on hidden UI buttons for security.

==========

89. DUPLICATE ACTION PROTECTION

==========

Prevent duplicate:

- Payments
- Reviews
- Reservations
- Webhooks
- Messages where appropriate

Use server-side validation and database constraints.

==========

90. COMPLETE SELLING FLOW

==========

SELLER:

Sell

↓

Add Photos

↓

Product Information

↓

Price

↓

Backend Calculates Posting Fee

↓

Meetup Location

↓

Preview

↓

Pay Posting Fee

↓

PayMongo QR / Payment

↓

PayMongo Confirms

↓

Backend Webhook

↓

Listing ACTIVE

↓

Buyer Finds Listing

↓

Buyer Messages Seller

↓

Chat

↓

Seller Reserves

↓

Face-to-Face Meetup

↓

Buyer Pays Seller Directly

↓

Seller Gives Item

↓

Seller Marks Sold

↓

Buyer Confirms

↓

Transaction Completed

↓

Review

==========

91. COMPLETE BUYING FLOW

==========

Buyer:

Home

↓

Explore

↓

Search

↓

Product

↓

View Seller

↓

Message Seller

↓

Chat

↓

Agree on Meetup

↓

Seller Reserves

↓

Meet Face-to-Face

↓

Buyer Pays Seller Directly

↓

Receive Item

↓

Confirm

↓

Transaction Completed

↓

Review

==========

92. COMPLETE WANTED FLOW

==========

Student:

Wanted

↓

Create Wanted Post

↓

Set Budget

↓

Publish

↓

Seller Sees Wanted Post

↓

Seller Responds

↓

Requester Gets Notification

↓

Chat

↓

Agree

↓

Meet

↓

Complete Transaction

↓

Mark Fulfilled

↓

Review

==========

93. COMPLETE PAYMENT FLOW

==========

SELLER:

Creates Listing

↓

Backend Calculates Fee

↓

₱1 / ₱5 / ₱10

↓

PayMongo Payment Created

↓

PayMongo QR / Payment Interface

↓

Seller Pays

↓

PayMongo

↓

Webhook

↓

ASP.NET Core API

↓

Verify Payment

↓

Payment = PAID

↓

Listing = ACTIVE

ACTUAL PRODUCT PAYMENT:

Buyer

↓

Meets Seller

↓

Pays Product Price Directly

↓

Seller

PasaBuy does NOT process the product price.

==========

94. PAYMENT DISCLAIMER

==========

Clearly communicate to users:

"PasaBuy only processes the listing fee required to publish a post. PasaBuy does not process or hold payment for products sold between students."

Also:

"Buyers and sellers are responsible for arranging and completing their own face-to-face transactions."

Safety message:

"Meet in a safe public location and inspect the item before paying."

==========

95. PAYMENT BUSINESS MODEL

==========

PasaBuy earns:

~~₱1~~

~~₱5~~

or

₱10

depending on the product's listing price.

PasaBuy does NOT take:

- Commission
- Percentage of sale
- Buyer payment
- Seller payment

This makes the business model simple:

SELLER WANTS TO POST

↓

PAY SMALL POSTING FEE

↓

LISTING GOES LIVE

==========

96. TESTING

==========

Test:

Authentication

Registration

Login

Logout

Password reset

Student verification

Listings

Images

Search

Filtering

Favorites

Chat

Wanted posts

Wanted responses

Reservations

Transactions

Reviews

Reports

Blocking

Notifications

Admin functions

==========

97. PAYMONGO TESTING

==========

Especially test:

₱1 → ₱1 fee

₱99 → ₱1 fee

₱100 → ₱5 fee

₱500 → ₱5 fee

₱999 → ₱5 fee

₱1,000 → ₱10 fee

₱10,000 → ₱10 fee

Test:

Successful payment

Failed payment

Cancelled payment

Duplicate webhook

Delayed webhook

Network interruption

User closing app during payment

Payment timeout

Payment status polling

Webhook retry

IMPORTANT:

A listing must NEVER become ACTIVE merely because the user returned to the application.

It must become ACTIVE only after backend-confirmed payment.

==========

98. ADMIN ANALYTICS REVENUE

==========

Admin revenue dashboard must calculate only actual PasaBuy posting-fee payments.

Example:

100 listings with ₱5 posting fees:

PasaBuy revenue:

₱500

Do not count:

Product selling prices

Agreed transaction prices

Cash exchanged between buyer and seller

as PasaBuy revenue.

==========

99. DEVELOPMENT ORDER

==========

Build in this order:

PHASE 1

Project architecture

PHASE 2

Database

PHASE 3

Authentication

PHASE 4

Student profiles

PHASE 5

Marketplace

PHASE 6

Sell/listing system

PHASE 7

PayMongo posting fee

PHASE 8

Chat

PHASE 9

Wanted posts

PHASE 10

Reservations and transactions

PHASE 11

Reviews

PHASE 12

Reports and blocking

PHASE 13

Admin web portal

PHASE 14

Analytics

PHASE 15

Security

PHASE 16

Testing

PHASE 17

UI polish

==========

100. FINAL USER EXPERIENCE

==========

The most important experience should be:

FIND

↓

CHAT

↓

AGREE

↓

MEET

↓

BUY/SELL

↓

REVIEW

The business model should be:

POST

↓

PAY SMALL LISTING FEE

↓

LISTING GOES LIVE

The actual product payment should remain:

BUYER ↔ SELLER

PasaBuy earns:

LISTING FEES

==========

101. IMPORTANT DEVELOPMENT RULES

==========

Do not create fake payment success.

Do not hard-code payment success.

Do not put PayMongo secrets in the mobile app.

Do not trust fee values from the mobile app.

Do not expose private user locations.

Do not expose passwords.

Do not allow unauthorized listing edits.

Do not allow duplicate reservations.

Do not allow duplicate reviews.

Do not activate unpaid listings.

Do not treat product sale prices as PasaBuy revenue.

Do not create a traditional product checkout.

Do not force buyers to pay PasaBuy for the actual product.

==========

102. FINAL EXPECTATION

==========

Build PasaBuy as a complete, polished, production-quality student marketplace.

The mobile application should feel:

Modern

Friendly

Fast

Clean

Safe

Professional

Easy to use

The admin web application should feel:

Professional

Data-driven

Organized

Secure

Easy to manage

Every screen should have:

- Proper loading state
- Proper error state
- Proper empty state
- Validation
- Responsive layout
- Consistent UI
- Good navigation

Create all required:

- .NET MAUI pages
- XAML

- C# ViewModels
- Models
- Services
- Repositories
- API controllers
- DTOs
- Database models
- MySQL schema
- Authentication
- Authorization
- PayMongo integration
- PayMongo webhook
- Chat system
- Wanted system
- Listing system
- Reservation system
- Transaction system
- Review system
- Notification system
- Report system
- Admin web portal
- Admin dashboard
- Analytics
- Security
- Validation
- Error handling

Make the code clean, modular, maintainable, and production-ready.

Do not leave major features as placeholders.

If a feature requires configuration such as PayMongo credentials, create the correct configuration structure and clearly identify where the developer must add the credentials without exposing or hard-coding secrets.

The final application must be a complete student marketplace called PASABUY.